FastAPI CRUD Application with Automated CI/CD, Databases, and Swarm Deployment

Project Overview

This project implements a **FastAPI**-based CRUD application for managing tasks. It is containerized using **Docker**, deployed with **Docker Swarm**, and integrates **PostgreSQL** and **Redis** for database and caching functionalities. Additionally, it features an automated **CI/CD pipeline** using **GitHub Actions**.

Key Features:

- **FastAPI REST API** with CRUD operations
- **PostgreSQL** as the relational database
- **Redis** for caching and session management
- **Docker containerization** for deployment
- **Docker Compose** for local development
- **Docker Swarm** for production-like orchestration
- **GitHub Actions CI/CD pipeline**
- **Automated testing** with Pytest

Project Breakdown & Requirements

1. FastAPI Application

The application is built with **FastAPI** and follows a clean architecture:

- Implements **CRUD** operations on a `tasks` resource.
- Uses **SQLAlchemy** for ORM with **PostgreSQL**.
- Integrates **Redis** for caching frequently accessed data.
- Implements **unit and integration tests** using **Pytest**.

API Endpoints:

- `POST /register` - Register a new user
- `POST /login` - Authenticate and get access token
- `POST /tasks/` - Create a new task
- `GET /tasks/` - Retrieve all tasks
- `GET /tasks/{task_id}` - Get a single task by ID
- `PUT /tasks/{task_id}` - Update a task
- `DELETE /tasks/{task_id}` - Delete a task

---

2. Containerization with Docker

Dockerfile:

Dockerfile is one of the most fundamental components in the Docker ecosystem and ensures that your project runs consistently across different environments (dev, test, production). When deploying your application running on port 8000 with Docker Swarm, the image created based on this Dockerfile is used.

The project is containerized using **Docker** to ensure portability and consistency across environments.

- Uses a **lightweight Python image** (e.g., `python:3.10-alpine`).
- Copies and installs dependencies from `requirements.txt`.
- Exposes the FastAPI application on **port 8000**.

Docker Compose (`docker-compose.yml`):

Defines services for:

- **FastAPI application**
- **PostgreSQL database** with persistent storage
- **Redis cache**

`docker-compose.yml`:

```
services:
postgres_db:
  image: postgres:15
  environment:
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: mysecretpassword
    POSTGRES_DB: fastapi_crud
  ports:
    - "5433:5432"
  volumes:
    - postgres_data:/var/lib/postgresql/data
  networks:
    - compose_backend
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 5s
    timeout: 5s
    retries: 5

redis_cache:
  image: redis:latest
  ports:
    - "6380:6379"
  networks:
    - compose_backend
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 5s
    timeout: 5s
    retries: 5

fastapi_app:
  build: .
  ports:
    - "8001:8000"
  environment:
    DATABASE_URL:
"postgresql://postgres:mysecretpassword@postgres_db:5432/fastapi_crud"
    REDIS_URL: "redis://redis_cache:6379/0"
```

```
  depends_on:
    postgres_db:
      condition: service_healthy
    redis_cache:
      condition: service_healthy
  networks:
    - compose_backend

test:
  build: .
  environment:
    DATABASE_URL:
"postgresql://postgres:mysecretpassword@postgres_db:5432/fastapi_crud"
    REDIS_URL: "redis://redis_cache:6379/0"
    PYTHONPATH: "/app"
  depends_on:
    postgres_db:
      condition: service_healthy
    redis_cache:
      condition: service_healthy
  networks:
    - compose_backend
  command: >
    bash -c "pip install httpx pytest pytest-asyncio &&
            cd /app &&
            python -m pytest -v tests/"
  volumes:
    - ./:/app
volumes:
postgres_data:
```

networks: compose_backend: driver: overlay

Creating and Starting a Container:
```bash
docker-compose up -d
```

Viewing Working Containers:

```
docker ps
```

To create images and start services:

```
docker-compose build
docker-compose up --build -d
```

Some commands I use:

```
docker logs [8aafd5a3966978969f6df37069ee19a7d31cdadb74478a56c01c76b2300840a1]
docker restart [fastapi-crud-app-redis_cache-1]
docker-compose down
docker-compose up --build -d
docker-compose logs -f
```

My Application Port:

```
curl localhost:8001
```

---

3. CI/CD with GitHub Actions

The project features an automated **CI/CD pipeline** using **GitHub Actions**.

Workflow Steps:

- **Lint & Test**: Run **black**, **flake8**, and **pytest**.
- **Build Docker Image**: Creates an image for the application.
- **Push to Docker Hub**: Uploads the image to **Docker Hub**.
- **Deploy to Production**: Deploys the container using **SSH and Docker Stack**.

  `ci-cd-pipeline.yml` GitHub Actions workflow:

  ```
  name: CI/CD Pipeline for FastAPI
  ```

on: push: branches: - dev pull_request: branches: - dev

jobs: build-and-test: runs-on: ubuntu-latest

```
services:
  postgres:
    image: postgres:15
    env:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: mysecretpassword
      POSTGRES_DB: fastapi_crud
    ports:
      - 5432:5432
    options: >-
      --health-cmd pg_isready
      --health-interval 10s
      --health-timeout 5s
      --health-retries 5

  redis:
    image: redis:latest
    ports:
      - 6379:6379
    options: >-
      --health-cmd "redis-cli ping"
      --health-interval 10s
      --health-timeout 5s
      --health-retries 5

steps:
  - name: Checkout repository
    uses: actions/checkout@v3
```

```
  - name: Set up Python
    uses: actions/setup-python@v3
    with:
      python-version: "3.11"

  - name: Install dependencies
    run: |
      python -m pip install --upgrade pip
      pip install -r requirements.txt
      pip install pytest pytest-asyncio httpx

  - name: Run Tests
    env:
      PYTHONPATH: ${{ github.workspace }}
      TESTING: "true"
    run: |
      pytest -v tests/
```

docker-build-and-push: needs: build-and-test runs-on: ubuntu-latest

```
steps:
  - name: Checkout repository
    uses: actions/checkout@v3

  - name: Log in to Docker Hub
    run: echo "${{ secrets.DOCKERHUB_PASSWORD }}" | docker login -u "${{
secrets.DOCKERHUB_USERNAME }}" --password-stdin

  - name: Build Docker image
    run: docker build -t hhilal/fastapi-crud-app:latest .

  - name: Push Docker image
    run: docker push hhilal/fastapi-crud-app:latest
```

deploy-to-swarm: needs: docker-build-and-push runs-on: ubuntu-latest if: github.ref == 'refs/heads/dev'

```
steps:
  - name: Checkout repository
    uses: actions/checkout@v3

  - name: Install SSH key
    uses: shimataro/ssh-key-action@v2
    with:
      key: ${{ secrets.SSH_PRIVATE_KEY }}
      known_hosts: unnecessary
      if_key_exists: replace

  - name: Adding Known Hosts
    run: |
      mkdir -p ~/.ssh
      ssh-keyscan -t rsa,ecdsa,ed25519 -p 22 ${{ secrets.SSH_HOST }} >>
```

```
~/.ssh/known_hosts
      shell: bash
  - name: Copy docker-stack.yml to server
    run: scp ./docker-stack.yml ${{ secrets.SSH_USERNAME }}@${{ secrets.SSH_HOST
}}:/tmp/docker-stack.yml

  - name: Deploy to Docker Swarm
    run: |
      ssh ${{ secrets.SSH_USERNAME }}@${{ secrets.SSH_HOST }} '
        # Pull the latest image
        docker pull hhilal/fastapi-crud-app:latest

        # Check if stack exists
        if docker stack ls | grep -q "fastapi_stack"; then
          echo "Updating existing stack..."
        else
          echo "Deploying new stack..."
          # Initialize swarm if not already done
          if ! docker info | grep -q "Swarm: active"; then
            docker swarm init
          fi
        fi

        # Deploy or update the stack
        docker stack deploy -c /tmp/docker-stack.yml fastapi_stack --with-registry-
auth

        # Bekle ve hizmetlerin başlamasını sağla
        sleep 30

        # Servislerin çalışıp çalışmadığını kontrol et
        echo "Deployed services:"
        docker stack services fastapi_stack
      '

  - name: Debug Services
    run: |
      ssh ${{ secrets.SSH_USERNAME }}@${{ secrets.SSH_HOST }} '
        echo "Checking PostgreSQL connection"
        docker run --rm --network fastapi_stack_backend postgres:15 pg_isready -h
postgres_db -p 5432

        echo "Checking Redis connection"
        docker run --rm --network fastapi_stack_backend redis redis-cli -h redis_cache
ping

        # Verify deployment
        echo "Service logs:"
        docker service logs fastapi_stack_postgres_db --tail 10
      '
```

````
##### Adding GitHub Secrets:
Go to your GitHub repository: Settings > Secrets and variables > Actions
Add the following secrets:

DOCKERHUB_USERNAME: Your Docker Hub username

DOCKERHUB_TOKEN: Your Docker Hub access token

SSH_HOST: The IP address of your Swarm manager

SSH_USERNAME: Your SSH username

SSH_PRIVATE_KEY: Your SSH private key


---

### 4. Deployment with Docker Swarm

The project is deployed using **Docker Swarm**, enabling scalability and high
availability.

#### Steps:
1. **Initialize Swarm**: `docker swarm init`
2. **Join Worker Nodes**: `docker swarm join ...`
3. **Deploy Application**: `docker stack deploy -c docker-stack.yml fastapi-app`

Example `docker-stack.yml`:
```yaml
version: "3.8"

services:
  postgres_db:
    image: postgres:15
    deploy:
      restart_policy:
        condition: on-failure
        max_attempts: 3
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: mysecretpassword
      POSTGRES_DB: fastapi_crud
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - backend
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "postgres", "-d", "fastapi_crud"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 10s
````
````

```
  redis_cache:
    image: redis:latest
    deploy:
      restart_policy:
        condition: on-failure
        max_attempts: 3
    networks:
      - backend
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 10s

  fastapi_app:
    image: hhilal/fastapi-crud-app:latest
    deploy:
      replicas: 2
      restart_policy:
        condition: on-failure
        delay: 10s
        max_attempts: 3
    ports:
      - target: 8000
        published: 8000
        protocol: tcp
        mode: ingress
    environment:
      DEPLOYMENT_TYPE: "swarm"
      DATABASE_URL:
"postgresql://postgres:mysecretpassword@postgres_db:5432/fastapi_crud"
      REDIS_URL: "redis://redis_cache:6379/0"
    networks:
      - backend

volumes:
  postgres_data:

networks:
  backend:
    driver: overlay
```

Other terminal commands I use:

```
docker service ls
docker service ps fastapi-app
curl http://localhost:8000
```

About Github

I used flake8, black, pylint, isort to make the code cleaner. I increased this value while Flake8 was fixing line 79. I wanted to stretch these points because they were very sensitive at some points. I also added code to the (.pre-commit-config.yaml) file. for example: .github/workflows/linter.yml:

```
name: Linter Check

on:
  pull_request:
    branches:
      - main
      - dev

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v3
        with:
          python-version: "3.11"

      - name: Install dependencies
        run: pip install flake8 black pylint isort

      - name: Run Flake8
        run: flake8 app/ --max-line-length=150 --ignore=E501 --count --statistics --
exit-zero

      - name: Run Black
        run: black --check --diff app/ || true

      - name: Run Pylint
        run: pylint app/ --disable=C0114,C0115,C0116,R0903,E0401,W0718 || true

      - name: Run isort
        run: isort --check --diff app/ || true
```

---

5. Virtual Machines & Networking

Connecting to Ubuntu from Terminal:

```
sudo apt update
sudo apt install openssh-server
```

For **Docker Swarm**, multiple VMs are set up using **VirtualBox**:

- **VM1 (Manager Node)**: `192.168.1.105`
- **VM2 (Worker Node)**: `192.168.1.110`

- **VM3 (Worker Node)**: `192.168.1.111`

For these VMs I used a Linux distribution, probably like Ubuntu Server 20.04/22.04. Docker installation on each virtual machine was performed as follows:

```
sudo apt update
sudo apt install -y apt-transport-https ca-certificates curl software-properties-
common

curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -

sudo add-apt-repository
"deb [arch=amd64] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable"

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io

sudo systemctl start docker
sudo systemctl enable docker

sudo usermod -aG docker $USER
```

I started a swarm cluster on the manager node. Commands used:

```
# On Manager Node
docker swarm init --advertise-addr 192.168.1.105

# On Worker Nodes: (I can't share it because it's confidential information.)
docker swarm join --token <TOKEN> 192.168.1.105:2377
```

Check the cluster status on the Manager node:

```
docker node ls
```

Then I transferred and deployed the stack file for the docker stack distribution:

```
nano docker-compose.yml
#I added my docker stack
#I think I made a mistake here
docker stack deploy -c docker-compose.yml fastapi-app
docker stack ls
docker stack services fastapi-app
docker stack ps fastapi-app
```

Network configuration between VMs is a critical step for a Docker Swarm cluster to function properly. Here is how I perform this configuration: Each VM is assigned a fixed IP address, allowing them to communicate with each other consistently:

```
sudo nano /etc/netplan/01-netcfg.yaml
```

```
network:
  version: 2
  renderer: networkd
```

```
  ethernets:
    enp0s8:
      dhcp4: no
      addresses:
        - 192.168.1.105/24
      gateway4: 192.168.1.1
      nameservers:
        addresses: [8.8.8.8, 8.8.4.4]
```

```
sudo netplan apply
sudo reboot
```

Finally, I tried entering from the terminal:

```
ssh vboxuser@192.168.1.105
```

Link: http://192.168.1.105:8000/docs

http://192.168.1.110:8000/docs

http://192.168.1.111:8000/docs

---

6. How the Workflow Works:

Workflow is triggered on every push to the dev branch

Docker image is created and pushed to Docker Hub

Swarm manager is connected via SSH

Latest image is pulled and stack is updated

Monitoring & Logging

- Logs can be viewed using `docker logs <container_id>`.

---

7. Manual testing with Swagger UI

You can access the screenshots I took during the manual testing in the Project_Doc file I added to the project.

---

8. Documentation & Best Practices

- **Security**: Secrets managed via **GitHub Secrets**.
- **Scalability**: Uses **Docker Swarm replicas**.
- **Resilience**: Includes health checks for containers.

Prerequisites

- Docker & Docker Compose installed
- Python 3.10+

Run Locally

```
git clone https://github.com/yourusername/fastapi-crud-app.git
cd fastapi-crud-app
```

```
docker-compose up --build
```

Deploy to Swarm

```
docker stack deploy -c docker-stack.yml fastapi-app
```

9. CI/CD Pipeline Execution

- Every **push to `dev` branch** triggers the pipeline.
- I preferred to open a branch for some of my specific developments, use it and then delete it.
- I did not merge the project into main because I received an error in the deploy section of the terminal. After resolving the error, I wanted to merge it and deliver main to you without any errors.

---

10. Common Errors and Solutions

1. Swarm Connection Issues: Error: `Error response from daemon: This node is not a swarm manager` Solution:

   ```
   # Check the Swarm status
   docker info | grep Swarm
   ```

Restart Swarm

docker swarm init --advertise-addr

```
2. Port Conflict:
Error: `Bind for 0.0.0.0:8000 failed: port is already allocated`
```bash
# Check which service is using the port
sudo netstat -tulpn | grep 8000

# Stop the running container/service
docker service update --publish-rm 8000:80 --publish-add 8001:80 myapp_webapp
```

3. Image Pull Failure: Error: `Error response from daemon: pull access denied, repository does not exist or may require authorization`

   ```
   # Log in to Docker Hub
   docker login
   ```

Specify the registry URL for a private registry

docker login your-registry.com
   ```

Add registry credentials in the stack configuration

````
4. GitHub Actions SSH Errors:
Error: `Permission denied (publickey)`
Solution:Ensure your SSH key uses the correct format
Verify that you have added the full private key to GitHub Secrets
Check the authorized_keys file on the target server
Confirm the SSH user has the required permissions

5. Stack Deployment Not Working:
Error: `service myapp_webapp: failed to create service: Error response from daemon`
Solution:
```bash
# Check service logs
docker service logs myapp_webapp

# Validate stack configuration
docker stack deploy --prune -c stack.yml myapp
````

Conclusion

This project covers **FastAPI application development**, **containerization**, **CI/CD automation**, and **Swarm deployment**, making it a great DevOps and backend development practice.

CONTRIBUTORS

- **Hatice Hilal AKSOY

````